

# Módulo 1

## Back-end com Node.js e Front-end com React

### Aula 01 — JavaScript sai do navegador

# Onde vamos chegar em 10 semanas

Uma aplicação **full stack** completa, feita por você:

- Um **back-end** que guarda e devolve dados (API)
- Um **banco de dados** de verdade
- Um **front-end** em React que consome essa API
- Código organizado, versionado no Git

Hoje é o primeiro tijolo.

# O mapa do módulo

| Semanas | O que construímos                |
|---------|----------------------------------|
| 1–3     | JavaScript fora do navegador     |
| 4–7     | O back-end: API + banco de dados |
| 8–9     | O front-end em React             |
| 10      | Tudo junto: projeto final        |

# Até agora: JS preso no navegador

```
document.querySelector("h1").textContent = "Olá!";  
alert("Oi!");
```

Esse código só funciona dentro de uma página.

E se eu quiser:

- Ler um arquivo do computador? ✗
- Guardar dados num banco? ✗
- Atender 1000 usuários ao mesmo tempo? ✗

# O motor por trás do navegador

O Chrome tem dentro dele um programa que lê e executa JavaScript:

**V8**

Ele não sabe o que é `document` .

Ele só sabe executar **JavaScript puro**.

## A ideia genial (2009)

"E se a gente tirar o V8 do navegador e deixar ele rodar **sozinho**?"

= Node.js 

# Mesma linguagem, ambientes diferentes

## — NAVEGADOR —

Motor V8

- + document, window
- + alert, fetch

✗ não lê arquivos

## — NODE.JS —

Motor V8

- + ler/escrever arquivo
- + criar servidor
- + acessar o sistema

✗ não tem document

for, if, funções, arrays: **idênticos** nos dois.

# A analogia do motor

O **V8** é o motor.

 No navegador → está num **carro de passeio**

 No Node → está num **trator**

Mesmo motor. Trabalhos diferentes.

# Quem usa Node.js

Netflix · Uber · LinkedIn · PayPal · NASA · praticamente toda startup

Não é ferramenta de curso.

É a ferramenta do seu **primeiro emprego**.

# Primeiro programa

`ola.js`

```
console.log("Olá, mundo do back-end!");
```

Terminal:

```
node ola.js
```

Sem HTML. Sem navegador. Sem `<script>` .

## O erro que vai acontecer

```
Error: Cannot find module '/caminho/ola.js'
```

Tradução: **"esse arquivo não existe aqui"**

Solução: rode `ls` (ou `dir`) e confira se o arquivo aparece.

Se não aparece → você está na pasta errada.

 **Ferramenta nova**

```
console.log()
```

## `console.log()` — o que é

**O que faz:** escreve uma mensagem no terminal

**Quando usar:** para **você** ver o que está acontecendo dentro do programa

```
console.log("texto");  
console.log(42);  
console.log({ nome: "Ana" }); // mostra o objeto inteiro  
console.log("a", "b", 42);    // vários, separados por vírgula
```

# A família do `console`

| Método                       | Para quê                            |
|------------------------------|-------------------------------------|
| <code>console.log()</code>   | mensagem normal                     |
| <code>console.error()</code> | erro (sai em vermelho no navegador) |
| <code>console.warn()</code>  | aviso                               |
| <code>console.table()</code> | array de objetos como <b>tabela</b> |

Experimente `console.table()` com uma lista.

Vai te salvar horas de depuração. ★

## **console.log** não é **return**

```
function somar(a, b) {  
  console.log(a + b);    // MOSTRA na tela  
}                        // e devolve undefined  
  
function somar(a, b) {  
  return a + b;          // DEVOLVE para quem chamou  
}
```

**console.log** é para **você** ver

**return** é para o **programa** usar

A confusão nº 1 de quem vem do Módulo 0.

# Ferramenta nova

## Template literals

# As crases: montando texto

```
const nome = "Maria";  
const idade = 28;  
  
// jeito antigo, com +  
"Olá, " + nome + ", " + idade + " anos"  
  
// jeito moderno ★  
`Olá, ${nome}, ${idade} anos`
```

**O que faz:** monta texto com variáveis dentro

**Quando usar:** sempre que misturar texto e variável

## Dentro de `${}` cabe qualquer conta

```
`Nasceu em ${2026 - idade}`
```

```
`Total: ${valor * quantidade}`
```

```
`Status: ${idade >= 18 ? "adulto" : "menor"}`
```

Não é só variável: é qualquer **expressão**  
(qualquer coisa que resulte num valor).

## ⚠ É crase, não aspas

```
'Olá, ${nome}'      // ✗ imprime literalmente ${nome}
`Olá, ${nome}`      // ✓
```

No teclado ABNT2: tecla ao lado do **P**  
(aperte a tecla e depois espaço)

## Recebendo dados de fora

```
const nome = process.argv[2];  
console.log(`Olá, ${nome}!`);
```

```
node saudacao.js Maria  
# Olá, Maria!
```

- | `argv[0]` | caminho do node |
- | `argv[1]` | seu arquivo |
- | `argv[2]+` | o que você digitou |

## `process` — o que é isso?

`process` é um objeto que o **Node** oferece com informações sobre a execução atual.

Ele não existe no navegador.

| Propriedade                  | O que é                         |
|------------------------------|---------------------------------|
| <code>process.argv</code>    | o que foi digitado no terminal  |
| <code>process.env</code>     | variáveis de ambiente (aula 06) |
| <code>process.exit(1)</code> | encerra o programa agora        |

## **process.argv** é sempre TEXTO

```
// node soma.js 10 3
const a = process.argv[2]; // "10" ← texto!
const b = process.argv[3]; // "3"

a + b // "103" 🧠 concatenou

Number(a) + Number(b) // 13 ✅
```

**Number()** converte texto em número.

Vamos usar isso o módulo inteiro.

## `process.exit(codigo)`

**O que faz:** encerra o programa **imediatamente**

**Quando usar:** quando faltou algo essencial  
e não faz sentido continuar

```
if (!nome) {  
  console.log("Uso: node script.js <nome>");  
  process.exit(1);  
}
```

`0` = terminou bem · qualquer outro = deu erro

# De script para projeto

```
npm init -y
```

Cria o **package.json** : a identidade do projeto.

- nome e versão
- lista de dependências
- comandos ( **scripts** )

# O package.json do curso

```
{
  "name": "meu-projeto",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  }
}
```

"type": "module" → habilita import / export

--watch → reinicia sozinho quando você salva ★

# Módulos: um arquivo por responsabilidade

saudacao.js

```
export function saudar(nome) {  
  return `Olá, ${nome}!`;  
}
```

index.js

```
import { saudar } from "./saudacao.js";  
console.log(saudar("Turma FPA"));
```

## 3 regras dos imports

1. `./` quando o arquivo é **seu**
2. Extensão `.js` é **obrigatória**
3. Só sai o que tem `export`

```
import { saudar } from "./saudacao.js";  
//           ↑           ↑  
//           ./ obrig.  .js obrig.
```

## E o `require`?

```
const x = require("./arquivo"); // ✗ jeito antigo  
import { x } from "./arquivo.js"; // ✓ nosso padrão
```

`require` = CommonJS, o sistema **antigo** do Node.

Você vai ver muito em tutoriais.

**Saiba reconhecer, mas escreva o moderno.**

## npm: 2 milhões de pacotes prontos

```
npm install chalk
```

```
import chalk from "chalk";  
console.log(chalk.green("✓ deu certo"));
```

Antes de escrever algo difícil:

**"alguém já resolveu isso?"**

## O que vai (e o que não vai) para o Git

|                                | Git?               |
|--------------------------------|--------------------|
| <code>package.json</code>      | ✅ é a receita      |
| <code>package-lock.json</code> | ✅ trava as versões |
| <code>node_modules/</code>     | ❌ <b>NUNCA</b>     |

`node_modules` pesa centenas de MB  
e volta inteira com `npm install`.

# Recapitulando

1. **Node.js** = JavaScript fora do navegador, com poderes de servidor
2. **package.json** = identidade do projeto · **node\_modules** nunca vai pro Git
3. **export** / **import** = dividir o código em arquivos

# Mão na massa 🚀

`aula-01/exercicios/README.md`

**Próxima aula:** o JavaScript que profissionais escrevem de verdade